Wine Quality Training Report

Preprocessing, Overfitting Controls, and Results from 26 Runs

October 26, 2026

I compared 26 neural-network runs using a dataset of 5,000 examples, 11 numeric features, and five quality classes. This report explains how I prepared the data, what I tried to reduce overfitting, and what happened in each experiment. The best observed test accuracy among models trained on all five classes was 44.80%, compared with 43.07% from always predicting Medium-Low. Most models still struggled with the rarer classes.

1 Preprocessing Methods

Dataset, Features, and Target Distribution

The source file, `wine_quality.csv`, contains 5,000 rows with 11 numeric predictors and one categorical target, `quality`. The inputs are fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, and alcohol. Quality is excluded from the feature matrix before splitting. The output class order is Low, Medium-Low, Medium, Medium-High, and High. Inspection of the current source file found no missing values and no exactly duplicated complete rows. I checked the dataset and found no missing values or duplicate rows, so neither required additional cleaning.

The class distribution is strongly imbalanced (Table 1). Medium-Low and Medium contribute 4,167 examples, or 83.34% of the complete dataset. High contributes only 50 examples. Consequently, a classifier can obtain an apparently substantial accuracy while failing to recognize most classes. In the original five-class test set, predicting Medium-Low for every row correctly labels 323 of 750 examples, giving a 43.07% majority baseline.

Table 1: Class counts for the ordinary all-class split. "Clean" denotes training after IQR filtering, before sampling.

Class	Full	Raw train	Clean	Val.	Test
Low	163	114	111	24	25
Medium-Low	2151	1506	1448	322	323
Medium	2016	1411	1363	303	302
Medium-High	620	434	414	93	93
High	50	35	35	8	7
Total	5000	3500	3371	750	750

Stratified Training, Validation, and Test Splits

I split the dataset into 70% training, 15% validation, and 15% testing, making sure each set kept approximately the same proportions of the five quality classes as the original dataset.The fixed data seed is distinct from the neural-network training seed, which controls parameter initialization, dropout masks, and minibatch order.

Splitting occurs before estimating outlier limits or scaling statistics. Validation guides checkpoint selection through cross entropy. Ordinary runs evaluate the test set after training, without using it for gradient updates or early stopping.

Training-Only IQR Outlier Filtering

I used the interquartile range (IQR) method to identify extreme training examples. For each feature, I calculated the lower and upper quartiles from the training data and allowed a margin of 1.5 times the IQR below and above them. I removed a training row if any feature was outside those limits. Validation and test

rows were kept, and neither set was used to calculate the limits.

For all-class training, this rule removes 129 of 3,500 rows (3.69%), leaving 3,371 examples. Removals by class are Low: 3/114 (2.63%); Medium-Low: 58/1,506 (3.85%); Medium: 48/1,411 (3.40%); Medium-High: 20/434 (4.61%); and High: 0/35. Logging these losses exposes whether cleaning disproportionately reduces a class. Filtering can reduce sensitivity to extreme feature values, but it can also discard legitimate observations. Its effect should therefore be treated as a modeling choice rather than proof that every removed row was invalid.

Scaling and Preprocessing Order

I used StandardScaler to put the numeric features on comparable scales. It was fitted on the cleaned training data before oversampling, then the same fitted scaler was applied to training, validation, and test data. Validation and test statistics were not used when fitting it.

Scaling makes feature magnitudes more comparable for network optimization and for SMOTE's neighbor distances. Fitting before sampling prevents duplicated or synthetic rows from changing the estimated means and scales. The usual order is: split; optionally exclude selected training classes; estimate IQR limits and filter training rows; fit the scaler; apply the chosen sampling procedure; transform the data; and train. Random oversampling occurs before transformation, whereas SMOTE operates on scaled training features. Training features and labels are shuffled together using data seed 42, preserving their pairing.

Imbalance Treatments

Random oversampling

Run 1 samples Low and High training rows with replacement until each class contains 400 examples. The cleaned pool contains 111 Low and 35 High rows, so 289 Low duplicates and 365 High duplicates are added. This increases training size from 3,371 to 4,025 while leaving the other training classes and all held-out distributions unchanged. Duplicates alter class representation without adding independently observed feature information.

SMOTE

Run 2 applies SMOTE after scaling, with five neighbors and random state 42, to raise Low and High to 400 examples each. Synthetic feature points are formed between same-class training examples. Validation and test data are neither oversampled nor used to create the synthetic rows. Unlike collecting new measurements, interpolation changes the available training representation without establishing that the synthesized observations correspond to independently measured wines.

Class-weighted loss

In run 4, I kept the cleaned training rows and used class-weighted cross entropy. Classes with fewer training examples received larger weights, using square-root inverse-frequency weights calculated from the training counts. Validation loss and early stopping remained unweighted so they could be compared with other runs. This changed the importance of training errors without generating new examples. Oversampling and class weights mainly address class imbalance rather than overfitting.

Target Encoding and Network Outputs

I initially represented quality labels as integer class indices. From run 23 onward, I used one-hot encoding for the quality labels while leaving the numeric inputs unchanged. The standard models still had five output nodes and used cross entropy as the loss function. Softmax was used to save class probabilities.

With hard class labels and no label smoothing, integer targets and one-hot targets give equivalent cross-entropy objectives. This change does not add information or correct the class imbalance. In practice, run 23 produced the same class predictions as run 22.

2 Strategies Used to Reduce Overfitting

L1, L2, and Elastic Net

I used cross entropy for training and added a penalty to the network's Linear-layer weights to discourage overfitting. Biases and learned PReLU slopes were not penalized. Adam's separate weight decay was left at zero so the same penalty was not applied twice. Most runs used L2 regularization with strength 0.0001. Runs 24–26 compared L1, L2, and Elastic Net at that strength.

L2 discourages large weights. L1 encourages smaller absolute weights and can make some weights close to zero. Elastic Net combines both penalties; I used a 50/50 mix. The same strength does not produce equal-sized penalties across these methods, so I also logged the actual penalties and weight statistics.

At the tested setting, L1 yields 43.87% test accuracy, while L2 and Elastic Net each yield 44.27%. L2 reproduces the earlier one-hot GELU checkpoint, and Elastic Net matches its test class predictions. These trials do not show a classification improvement from switching penalties, but they also do not evaluate other strengths or mix ratios. Evaluation losses and the checkpoint monitor exclude all weight penalties.

Dropout

Dropout is inserted after each hidden activation. Probabilities 0.20, 0.10, and 0.05 are tested. Randomly removing hidden activations during minibatch training is intended to reduce dependence on individual activation paths. Dropout is disabled for end-of-epoch training metrics, validation, and final test evaluation. Consequently, the logged training and validation curves use the same evaluation mode.

Run 5's dropout of 0.20 improves validation accuracy and lowers validation cross entropy relative to the no-dropout baseline, but its test macro F1 decreases. It recovers 83.90% of Medium-Low examples while recovering only 17.55% of Medium examples. This illustrates why a lower validation loss or higher aggregate accuracy does not ensure balanced class recognition. Lower dropout probabilities do not provide a consistent improvement over no dropout in the early experiments.

Early Stopping and Checkpoint Restoration

Each epoch ends with unweighted training and validation cross-entropy evaluation. A checkpoint is accepted when validation loss decreases by more than 0.0001 relative to the best qualifying value. Training stops after the configured number of consecutive non-improving epochs, then restores the accepted checkpoint. The usual epoch limit is 100. Patience values 4, 2, 10, and 5 are explored; the final setting is 5.

For the single-32 ReLU network, patience 2 trains 17 epochs and restores epoch 15 (run 16). Patience 10 trains 34 epochs and restores epoch 24 (run 18). Patience 5 trains 29 epochs and restores that same epoch-24 model (run 19), saving five epochs without changing final predictions. Moving from patience 2 to the later checkpoint slightly improves validation cross entropy but lowers validation and test accuracy. Early stopping selects for its monitor, rather than jointly maximizing every classification metric.

Network Capacity

The hidden layouts tested are [32,16], [64,32], [32], and [16]. All standard networks accept 11 features and emit five logits. Parameter counts, including Linear biases, are 997, 3,013, 549, and 277 respectively. The single-32 PReLU version has 550 parameters because it learns one slope. The two-output [64,32] diagnostic has 2,914 parameters.

Reducing capacity can constrain memorization, but making the model too small can also limit its ability to represent useful relationships. In the matched all-class comparison, the single-32 model reaches 44.80% test accuracy, compared with 39.73% for [64,32] and 40.80% for [16]. These observations support the single-32 layout under the tested seed and stopping settings; they do not establish a universal optimum.

Optimization, Activation, and Evaluation Safeguards

I used Adam with shuffled minibatches of 64 and a fixed learning rate of 0.001. Run 20 tried 0.0005. I kept He normal initialization and zero biases across the activation comparisons so the initial Linear weights stayed comparable.

PReLU learns one negative slope per hidden layer, initialized to 0.25; the restored single-layer slope is approximately 0.2198. GELU uses its exact, non-approximated form. Adam, activation changes, and initialization are optimization/modeling choices rather than independently demonstrated overfitting remedies in this assignment.

3 Experiments and Their Outcomes

How to Read the Results

Tables 2 and 3 document all 26 original runs. Input dimension is 11 throughout; output dimension is five except in runs 14 and 15. Training seed is 42 initially and 43 from run 12 onward; the data seed remains 42. Regularization is L2 with strength 0.0001 unless indicated otherwise. Run 15 used no weight penalty. Accuracy and recall are percentages, macro F1 is on the 0–1 scale, and cross entropy uses natural logarithms. Missing evaluations are marked by a dash rather than zero.

Table 2: Configuration register for all 26 runs. Hidden layers omit the common 11-input and five-output dimensions; runs 14 and 15 have two outputs. Data seed is 42 and batch size is 64 throughout. P is early-stopping patience; S is training seed. ROS means random oversampling, OH means one-hot targets, EN means Elastic Net, and Exclude means Medium-High/High excluded from training. All nonzero penalty strengths are 0.0001; EN has L1 ratio 0.5.

Run	Hidden	Activation	Dropout	LR	P	S	Balancing	Penalty	Targets	Scope
1	32/16	ReLU	0	0.001	4	42	ROS	L2	Index	All
2	32/16	ReLU	0	0.001	4	42	SMOTE	L2	Index	All
3	32/16	ReLU	0	0.001	4	42	None	L2	Index	All
4	32/16	ReLU	0	0.001	4	42	Weights	L2	Index	All
5	32/16	ReLU	0.2	0.001	4	42	None	L2	Index	All
6	32/16	ReLU	0.1	0.001	4	42	None	L2	Index	All
7	32/16	ReLU	0.05	0.001	4	42	None	L2	Index	All
8	32/16	ReLU	0.05	0.001	4	42	None	L2	Index	Exclude
9	32/16	ReLU	0.05	0.001	2	42	None	L2	Index	Exclude
10	64/32	ReLU	0.05	0.001	2	42	None	L2	Index	Exclude
11	64/32	ReLU	0.05	0.001	2	42	None	L2	Index	Exclude
12	64/32	ReLU	0.05	0.001	2	43	None	L2	Index	Exclude
13	64/32	ReLU	0.05	0.001	2	43	None	L2	Index	All
14	64/32	ReLU	0.05	0.001	2	43	None	L2	Index	2-class
15	64/32	ReLU	0	0.001	–	43	Subset	none	Index	64 only
16	32	ReLU	0.05	0.001	2	43	None	L2	Index	All
17	16	ReLU	0.05	0.001	2	43	None	L2	Index	All
18	32	ReLU	0.05	0.001	10	43	None	L2	Index	All
19	32	ReLU	0.05	0.001	5	43	None	L2	Index	All
20	32	ReLU	0.05	0.0005	5	43	None	L2	Index	All
21	32	PReLU	0.05	0.001	5	43	None	L2	Index	All
22	32	GELU	0.05	0.001	5	43	None	L2	Index	All
23	32	GELU	0.05	0.001	5	43	None	L2	OH	All
24	32	GELU	0.05	0.001	5	43	None	L1	OH	All
25	32	GELU	0.05	0.001	5	43	None	L2	OH	All
26	32	GELU	0.05	0.001	5	43	None	EN	OH	All

All summary scores are computed at the restored checkpoint, except the memorization check, which reports its stopping-state training metrics. Run 4 additionally records weighted cross entropy, but the tables use unweighted evaluation loss. Precision is the fraction of a class's predictions that are correct; recall is the fraction of its true examples recovered. Macro F1 averages class-specific F1 equally, whereas accuracy counts every row equally. This distinction matters when two classes dominate the sample count.

Table 3: Restored-model outcomes. Accuracies are percentages; cross entropy (CE) and macro F1 use their original scales. E/B is epochs trained / best epoch. Run 14 reports original two-class validation only; its separate later five-class test accuracy is 43.87%. Run 15 reports training-only metrics, with no best validation epoch. The five-class test majority baseline is 43.07% accuracy and 0.1204 macro F1.

Run	Train acc.	Val. CE	Val. acc.	Val. macro F1	Test CE	Test acc.	Test macro F1	E/B
1	43.11	1.2605	40.27	0.1719	1.2544	41.73	0.1777	18/14
2	47.18	1.2531	39.47	0.1751	1.2573	42.27	0.2003	32/28
3	46.99	1.1793	41.33	0.1831	1.1703	42.40	0.1840	19/15
4	44.56	1.2459	39.20	0.1743	1.2471	40.93	0.1828	20/16
5	44.56	1.1529	43.47	0.1604	1.1531	43.20	0.1641	37/33
6	45.30	1.1626	42.13	0.1773	1.1623	40.53	0.1716	22/18
7	45.95	1.1694	40.93	0.1672	1.1634	42.13	0.1742	24/20
8	47.54	1.3896	41.07	0.1809	1.4122	40.80	0.1792	5/1
9	47.54	1.3896	41.07	0.1809	1.4122	40.80	0.1792	3/1
10	49.69	1.5667	39.07	0.1693	1.4729	45.73	0.1983	3/1
11	49.69	1.5667	39.07	0.1693	1.4729	45.73	0.1983	3/1
12	47.60	1.3221	41.60	0.1737	1.3924	38.00	0.1602	3/1
13	46.54	1.1834	42.13	0.1821	1.1877	39.73	0.1733	9/7
14	58.70	0.6953	53.12	0.5312	–	–	–	9/7
15	100.00	–	–	–	–	–	–	292/–
16	44.44	1.1717	42.80	0.1852	1.1871	44.80	0.1935	17/15
17	45.68	1.1805	42.13	0.1799	1.1648	40.80	0.1745	46/44
18	45.65	1.1703	42.67	0.1823	1.1809	43.87	0.1858	34/24
19	45.65	1.1703	42.67	0.1823	1.1809	43.87	0.1858	29/24
20	45.21	1.1708	42.80	0.1843	1.1828	43.73	0.1879	44/39
21	45.39	1.1650	43.60	0.1857	1.1770	42.80	0.1797	29/24
22	45.36	1.1717	43.20	0.1852	1.1835	44.27	0.1869	29/24
23	45.36	1.1717	43.20	0.1852	1.1835	44.27	0.1869	29/24
24	45.27	1.1714	42.80	0.1832	1.1830	43.87	0.1850	29/24
25	45.36	1.1717	43.20	0.1852	1.1835	44.27	0.1869	29/24
26	45.36	1.1716	42.93	0.1840	1.1832	44.27	0.1869	29/24

Sampling and Weighted Loss: Runs 1–4

Run 1. Random oversampling, with [32,16] hidden units and no dropout, reaches 40.27% validation and 41.73% test accuracy. Test macro F1 is 0.1777. Training stops at epoch 18 and restores epoch 14. Low and High duplication does not recover any rare-class test examples. Its held-out accuracy is below the unsampled baseline.

Run 2. SMOTE uses the same layout and training defaults. It trains 32 epochs, restoring epoch 28, and reaches 39.47% validation and 42.27% test accuracy. Test macro F1 rises to 0.2003, compared with 0.1840 without sampling. One Low test example is recovered (4% recall), but Medium-High and High recall remain zero. Validation accuracy and macro F1 are below the unsampled baseline, so the test macro-F1 increase is not a consistent overall improvement.

Run 3. With no oversampling and no dropout, validation accuracy is 41.33% and test accuracy is 42.40%, with macro F1 0.1840. The model trains 19 epochs and restores epoch 15. This is a better accuracy reference than random oversampling, but it remains below the 43.07% majority baseline. All three rare-class test recalls are zero.

Run 4. Square-root inverse-frequency class weights reduce validation accuracy to 39.20% and test accuracy to 40.93%. Test macro F1 is 0.1828; the best checkpoint is epoch 16 after 20 epochs. One of 93 Medium-High test examples is correctly classified, but Low and High recall remain zero. The tested weights do not resolve the imbalance.

Dropout Trials: Runs 5–7

Run 5. Dropout 0.20 reaches 43.47% validation and 43.20% test accuracy, restoring epoch 33 after 37 epochs. Validation cross entropy falls to 1.1529, but test macro F1 falls to 0.1641. Medium-Low recall rises to 83.90%, while Medium recall falls to 17.55%. Its aggregate score therefore masks a large common-class recall imbalance.

Run 6. Dropout 0.10 restores epoch 18 after 22 epochs. Validation accuracy is 42.13%, test accuracy is 40.53%, and test macro F1 is 0.1716. Test accuracy and macro F1 are lower than the no-dropout baseline, despite a somewhat higher validation accuracy. It does not provide a consistent improvement.

Run 7. Dropout 0.05 restores epoch 20 after 24 epochs, reaching 40.93% validation and 42.13% test accuracy. Test macro F1 is 0.1742. Medium-Low recall is 70.90%, versus 28.81% for Medium. Neither validation nor test accuracy improves over run 3, although this dropout setting is retained for later comparisons.

Exclusions, Patience, Capacity, and Seeds: Runs 8–13

Run 8. Removing Medium-High and High from training retains the five-output [32,16] model. With patience 4, the run trains five epochs and restores epoch 1. Validation accuracy is 41.07% and test accuracy is 40.80%, with macro F1 0.1792. Removing these classes does not improve test accuracy relative to run 7.

Run 9. Patience is reduced to 2 in the same excluded-class setting. Training stops after three epochs and restores the same epoch-1 checkpoint. Final scores and predictions match run 8. This saves two epochs but does not change the fitted result.

Run 10. The excluded-class model is enlarged to [64,32], retaining dropout 0.05 and patience 2. It again trains three epochs and restores epoch 1. Test accuracy rises to 45.73% and macro F1 to 0.1983, while validation accuracy falls to 39.07%. The strongest headline test accuracy therefore accompanies a weaker validation result and a different training-class scope from the all-class models.

Run 11. A repeat of run 10 uses the same training seed 42 and data seed 42. It reproduces the same scores, predictions, three-epoch stopping point, and epoch-1 checkpoint. This demonstrates reproducibility with a fixed seed, not stability across seeds.

Run 12. Only the training seed changes from 42 to 43 in the excluded-class [64,32] setting. Test accuracy falls to 38.00% and macro F1 to 0.1602, while validation accuracy becomes 41.60%. It restores epoch 1 after three epochs. This large change exposes sensitivity to initialization and stochastic training choices, although both runs stop very early.

Run 13. All five training classes are restored while retaining the [64,32] network, dropout 0.05, patience 2, and training seed 43. The model trains nine epochs, restoring epoch 7. Validation accuracy is 42.13% and test accuracy is 39.73%, with macro F1 0.1733. It improves both accuracies relative to run 12 and supplies the all-class large-network reference for the later capacity trials.

Diagnostic Tasks: Runs 14–15

Run 14. The two-output Medium-Low-versus-Medium network trains on 2,811 cleaned examples and validates on 625 matching examples. Its [64,32] hidden layout restores epoch 7 after nine epochs. Validation accuracy is 53.12% and macro F1 is 0.5312, versus a 51.52% majority baseline. This is only a small observed separation gain in the isolated two-class task.

A later evaluation of the saved two-output model on all 750 original test rows reaches 43.87% accuracy, balanced accuracy 21.08%, and macro F1 0.1914. On the 625 common-class rows alone, accuracy is 52.64%, compared with a 51.68% majority baseline. All 125 rows from the other classes are errors. These later scores are distinct from the original diagnostic's validation-only metrics and are not five-output cross-entropy results.

Run 15. The unregularized [64,32] two-output network memorizes a balanced subset of 64 examples. Perfect training accuracy first occurs at epoch 111; the stopping criteria are satisfied at epoch 292, with training cross entropy 0.00965 and accuracy 100%. There are no validation or test results. This supports

the implementation's ability to fit that selected subset, but does not establish generalization, useful held-out signal, or the cause of weak full-dataset results.

Single-Layer Capacity and Patience: Runs 16–19

Run 16. A single hidden layer of 32 ReLU units, dropout 0.05, patience 2, and all five training classes produces 42.80% validation and 44.80% test accuracy. Test macro F1 is 0.1935. It trains 17 epochs and restores epoch 15. This improves both accuracies over the matched [64,32] reference. Medium-Low and Medium recalls are 62.54% and 44.37%; the other test recalls remain zero.

Run 17. Reducing the single layer to 16 units trains 46 epochs and restores epoch 44. Validation accuracy is 42.13%, test accuracy is 40.80%, and macro F1 is 0.1745. Neither common-class recall improves over run 16. Lower capacity therefore does not automatically yield better generalization in this setting.

Run 18. Restoring the single-32 layout and increasing patience to 10 trains 34 epochs and restores epoch 24. Validation cross entropy improves slightly from run 16's 1.1717 to 1.1703, but validation accuracy becomes 42.67% and test accuracy 43.87%. Test macro F1 is 0.1858. Medium-Low recall rises to 68.11% as Medium recall falls to 36.09%.

Run 19. Patience 5 trains 29 epochs and restores the same epoch-24 checkpoint as patience 10. Final scores and predictions match run 18 exactly. This improves training efficiency by five epochs without improving predictive performance.

Learning Rate, Activation, and Targets: Runs 20–23

Run 20. Reducing Adam's learning rate to 0.0005 retains patience 5 and the single-32 ReLU layout. Training extends to 44 epochs and restores epoch 39. Validation accuracy is 42.80%, test accuracy is 43.73%, and test macro F1 is 0.1879. Medium recall rises from 36.09% to 40.73%, while Medium-Low recall falls from 68.11% to 63.47%. This redistributes common-class recall rather than clearly improving both.

Run 21. Learning rate 0.001 is restored and PReLU replaces ReLU. The run trains 29 epochs and restores epoch 24. Validation accuracy improves to 43.60% and cross entropy to 1.1650, but test accuracy falls to 42.80% and macro F1 to 0.1797. Medium recall falls to 32.78%. The validation gain does not carry over to test classification.

Run 22. GELU replaces ReLU, with the other settings matched to run 19. Validation accuracy is 43.20%, test accuracy is 44.27%, and macro F1 is 0.1869. The run trains 29 epochs and restores epoch 24. Compared with ReLU, this is three additional correct test predictions; validation cross entropy is slightly worse. The observed change is small and comes from one training seed.

Run 23. One-hot quality targets replace integer targets while GELU and other settings are unchanged. Training again stops at epoch 29 and restores epoch 24. Validation accuracy is 43.20% and test accuracy is 44.27%. Class predictions and scores match run 22. The equivalent label representation does not improve classification or rare-class recall.

L1, L2, and Elastic Net: Runs 24–26

Run 24. L1 replaces L2 with strength 0.0001. The single-32 GELU network retains one-hot targets, dropout 0.05, and patience 5. It restores epoch 24 after 29 epochs. Validation accuracy is 42.80%, test accuracy is 43.87%, and macro F1 is 0.1850. Validation cross entropy is slightly lower than the L2 reference, but both classification accuracies are lower. No rare-class test examples are recovered.

Run 25. The L2 rerun, at the same strength, exactly reproduces the previous one-hot GELU checkpoint from run 23. Validation accuracy is 43.20%, test accuracy is 44.27%, and macro F1 is 0.1869. It restores epoch 24 after 29 epochs and serves as the direct reference for the penalty comparison.

Run 26. Elastic Net uses regularization strength 0.0001 and a 50/50 L1–L2 mix. It restores epoch 24 after 29 epochs. Validation accuracy is 42.93%; test accuracy and macro F1 match L2 at 44.27% and 0.1869.

Its test class predictions also match L2, so the tested strength and mix do not improve classification.

What Worked and What Did Not

The most useful observed changes are the single-32 layout's improvement over the matched large all-class model, the epoch savings from patience 5 versus 10, and the small GELU accuracy gain over ReLU. PReLU improves validation cross entropy and accuracy but worsens test accuracy. SMOTE increases test macro F1 and recovers one Low example, but does not improve validation performance or test accuracy over no sampling. These mixed outcomes show that no single aggregate metric captures all objectives.

Most trials do not solve the central class-recognition problem. No saved test evaluation correctly classifies High. Run 2 recovers one Low example, while run 4 recovers one Medium-High example. Most other five-output runs have zero recall for all three rarer classes. High test support is only seven, which also limits the stability of class-specific estimates. Table 4 illustrates how common-class recall changes without a large overall accuracy improvement.

Table 4: Selected common-class test recalls and overall accuracy (%).

Run / setting	Medium-Low recall	Medium recall	Accuracy
5 / dropout 0.20	83.90	17.55	43.20
16 / patience 2	62.54	44.37	44.80
19 / patience 5	68.11	36.09	43.87
20 / lower LR	63.47	40.73	43.73
22 / GELU	69.66	35.43	44.27
26 / Elastic Net	69.66	35.43	44.27

Among models trained on all five classes, run 16 has the highest observed test accuracy at 44.80%, representing 336 correct examples versus 323 for the majority baseline. The 45.73% score from runs 10 and 11 comes from an excluded-class model with only 39.07% validation accuracy. Its seed-43 counterpart falls to 38.00% test accuracy. The largest test score alone is therefore insufficient evidence for selecting a robust all-class classifier.